

資料探勘hw2

一、前言

報告中會大量使用專案中的腳本，詳細的程式碼可直接到專案中，進行查看，以下為專案的結構，可到github進行查看。

https://github.com/rsktbob/data_analysis_hw2

```
.
├── data/                                # Processed data for training and
evaluation
│   ├── holdout/                        # 80/20 split data
│   └── cross_validation/              # 5-fold CV split data
├── models/                             # Saved SVM models
│   ├── holdout/                       # 使用data/holdout/train.scale訓練
│   └── cross_validation/              # 使用data/cross_validation/train_fold*
├── merge_data.py                       # Merges pokemon stats with combat records
├── data_transform.py                  # 資料前處理
├── feature_analysis.py                # 找尋重要的特徵
├── split_data.py                     # Data splitting (Holdout & CV)
├── train_model.py                    # train model (Holdout & CV)
├── predict_model.py                  # Prediction and metric calculation
├── roc_plot.py                       # ROC curve generation and AUC calculation
├── pokemon.csv                       # Original Pokémon dataset
├── combats.csv                       # Original combat records
├── merged_data.csv                   # 將pokemon.csv和combats.csv合併的資料
├── pokemon_data.txt                  # 經過data_transform, 只保留重要的特徵
├── pokemon_data.scale                # 經過svm-scale
├── 資料探勘hw2.pdf                   # 完整報告
└── svm-*.exe                         # LIBSVM executables (Windows)
```

二、問題一

Question 1. (20%) 請從寶可夢資料庫中，先分析各個 attribute，決定哪些可能是重要的特徵值。例如，你可以觀察寶可夢屬性的分布、單一屬性與雙屬性的影響，以及各項數值型屬性（如 HP、Attack、Defense 等）的平均值與標準差，藉此判斷哪些因素可能會對對戰結果造成影響，也可透過例如對戰的初始血量差距或其他的方法建立新的特徵值。請敘述你使用的分析方法，並說明你為什麼認為某些屬性值較重要。

此題透過觀察寶可夢的屬性分布、單/雙屬性影響、各項數值型屬性的統計量與差距，並結合 Filter Method 與 SVM 特性，得出最終的特徵選擇結論。

(一) 單屬性和雙屬性的影響

透過統計 5 萬場戰鬥紀錄，觀察「屬性多寡」對戰鬥結果的影響：

- 單一屬性 (Single Type) 平均勝率:46.58%
- 雙屬性 (Dual Type) 平均勝率:53.35%

分析結論：

屬性是寶可夢重要的特徵，彈藥選擇單屬性還是雙屬性保留就是一個值得思考的問題，透過戰鬥紀錄發現擁有雙屬性的寶可夢在勝率上具有約 6.7% 的絕對優勢，這可能是同時能透過雙屬性互相彌補防禦弱點。因此我選擇保留了雙屬性作為特徵。

屬性是類別型資料，需要經過one hot encoding，這裡的總共有18種屬性，在表達一隻寶可夢時選擇使用36維的話會造成維度爆炸，這裡選擇將一隻寶可夢的雙屬性直接做相加，假設有一隻草、戰鬥屬性的寶可夢，可表示成1100...0，第一位和第二位分別是草、戰鬥，並進一步地將第二隻寶可夢也表示成這種形式，並進行相減

(二) 傳說寶可夢與世代的影響

除了屬性之外，我們也針對「是否為傳說寶可夢 (Legendary)」以及「世代 (Generation)」進行了勝率與相關性分析

- 傳說寶可夢 (Legendary) 平均勝率:77.91%
- 一般寶可夢 (Non-Legendary) 平均勝率:47.62%
- 世代差距 (Diff_Generation) 與勝負的皮爾森相關係數:0.0147

分析結論：

傳說寶可夢具備統治級優勢：傳說寶可夢的勝率比一般寶可夢高出了驚人的 30.30%。即便不看基礎能力值，傳說寶可夢可能具有隱藏優勢(例如招式池較佳)。這是一個極強的特徵，必須保留，並使用diff去將原本要輸入寶可夢1是否為傳說和寶可夢是否為傳說，只要輸入一個，來降低特徵維度。

世代對勝負幾乎無影響：世代差距與對戰勝負的線性相關係數僅有 0.0147，趨近於零。代表不論對手是第幾世代的寶可夢，都不會實質影響戰鬥勝負(戰鬥力多半已完全反映在六圍數值與神獸標籤上)。因此世代特徵可以安全刪除。

(三) 數值型特徵分析

透過統計戰鬥紀錄，分析資料中的數值型屬性，並透過正規化(Z-Score)消除不同量綱影響，使各能力值可在相同標準下進行比較。

屬性	勝利方平均 Z-Score	落敗方平均 Z-Score	標準差
Speed	+0.519	-0.464	0.984
Attack	+0.276	-0.247	0.524

Sp. Atk	+0.276	-0.236	0.501
Sp. Def	+0.276	-0.147	0.312
HP	+0.276	-0.135	0.286
Defense	+0.276	-0.052	0.110

分析結論:

速度 (Speed) 是一切的王者:勝利方的平均速度比落敗方足足高出了近 1 個完整的標準差 (0.984) ! 這印證了寶可夢「先下手為強」的機制, 速度是最決定性的因素, 其他的屬性也有不錯的結果, 但是防禦力差距只有0.110, 這代表可能防禦力不是很好的特徵, 因此我選擇保留替除了防禦力所有特徵, 但如果直接將寶可夢1和寶可夢2的屬性直接輸入, 會導致維度爆炸, 為了減輕計算的雜訊, 這裡選擇使用兩隻寶可夢數值相減作為特徵

(四) 選用的特徵

經過上述分析, 我選用了以下這些特徵作為svm分析

1. diff_bug
2. diff_dark
3. diff_dragon
4. diff_Electric
5. diff_Fairy
6. diff_Fighting
7. diff_Fire
8. diff_Flying
9. diff_Ghost
10. diff_Grass
11. diff_Ground
12. diff_Ice
13. diff_Normal
14. diff_Poison
15. diff_Psychic
16. diff_Rock
17. diff_Steel
18. diff_Water

======(以上是屬性資料)

19. diff_Legend
20. diff_Speed
21. diff_Attack
22. diff_SpAtk
23. diff_SpDef
24. diff_HP

三、問題二

Question 2. (30%) Hold-out

請將對戰資料的 80% 作為 training set、20% 作為 test set。資料分割方式不限(例如 random sampling 或依資料順序切分)，但需注意 class imbalance 的問題。請至少嘗試五種不同的參數組合，並透過比較以下 evaluation metrics 來選擇你嘗試的參數組合裡面最佳的參數: Confusion Matrix, Accuracy, Precision, Recall (TPR), FPR, TNR 與 FNR，接著請具體說明：參數選擇的過程與依據以及每組參數訓練後模型的 confusion matrix 與各項 evaluation metrics 的數值

(一) 資料切割

使用sklearn中的train_and_test函數進行切割，並使用分層抽樣確保訓練集和資料集中的正樣本和負樣本是一樣的，以下為實現的程式碼。

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
dump_svmlight_file(X_train, y_train, os.path.join(output_dir, 'holdout', 'train.scale'))
dump_svmlight_file(X_test, y_test, os.path.join(output_dir, 'holdout', 'test.scale'))
```

(二) 訓練參數的選擇

本訓練使用以下5種參數進行訓練，並說明為什麼這樣設計的理由。

參數	-s	-t	-c	-g	設計理由
參數1	-s 0	-t 0	-c 1	無	使用linear kernel作為 Baseline。測試寶可夢對戰的特徵是否在原維度就具備良好的線性可分性
參數2	-s 0	-t 2	-c 1	-g 0.1	RBF 的基礎表現，使用適中的c與g看模型的表現
參數3	-s 0	-t 2	-c 10	-g 0.1	提高 C 值，減少訓練集的容錯率，試圖捕捉更多決策邊界的細節
參數4	-s 0	-t 2	-c 100	-g 0.01	將C開到非常大 (100) 以強制模型分類正確，但同時將`gamma`縮小至`0.01`來擴大個別樣本的作用範圍，避免過度擬合產生的陡峭決策邊界
參數5	-s 0	-t 2	-c 10	-g 1	將C降低到10，並將g調得很大，讓各點的影響範圍變積極小，測試是否會發生 Overfitting

各-s保持0，因為是進行分類任務，本次只有參數1使用linear kernel，其餘皆使用RBF kernel。

(三) 實驗結果

參數	模型表現
參數1	<pre> --- Model 1 Holdout Test Set Evaluation --- Confusion Matrix: [[4287 433] [484 4796]] Accuracy: 0.9083 Precision: 0.8986 Recall: 0.9083 FPR: 0.0917 TNR: 0.9083 FNR: 0.0917 </pre>
參數2	<pre> --- Model 2 Holdout Test Set Evaluation --- Confusion Matrix: [[4229 491] [442 4838]] Accuracy: 0.9067 Precision: 0.9054 Recall: 0.8960 FPR: 0.0837 TNR: 0.9163 FNR: 0.1040 </pre>
參數3	<pre> --- Model 3 Holdout Test Set Evaluation --- Confusion Matrix: [[4233 487] [455 4825]] Accuracy: 0.9058 Precision: 0.9029 Recall: 0.8968 FPR: 0.0862 TNR: 0.9138 FNR: 0.1032 </pre>

參數4	<pre> --- Model 4 Holdout Test Set Evaluation --- Confusion Matrix: [[4262 458] [441 4839]] Accuracy: 0.9101 Precision: 0.9062 Recall: 0.9030 FPR: 0.0835 TNR: 0.9165 FNR: 0.0970 </pre>
參數5	<pre> --- Model 5 Holdout Test Set Evaluation --- Confusion Matrix: [[4154 566] [532 4748]] Accuracy: 0.8902 Precision: 0.8865 Recall: 0.8801 FPR: 0.1008 TNR: 0.8992 FNR: 0.1199 </pre>

(四) 總結

五組模型中，Model 4 表現最佳($t=2$, $c=10$, $g=0.01$)，其準確率(Accuracy)達到 **0.9101**，同時在 Precision(0.9062)與 Recall(0.9030)之間維持良好平衡。此外，其 False Positive Rate($FPR = 0.0835$)亦為所有模型中較低者，顯示其分類能力較為穩定且泛化能力較佳。

相較之下，Model 5 的整體表現最差，Accuracy 僅為 **0.8902**，且 FPR 上升至 0.1008，顯示誤判情況較為嚴重。

其餘 Model 1、Model 2 與 Model 3 的表現相近，Accuracy 約落在 0.905~0.908 之間，屬於中等且穩定的結果。

四、問題三

Question 3. (30%) Cross-validation

請將上一題的 training set 分成五個等份，並進行 5-fold cross validation。請沿用上一題中所使用的五種參數組合，分別對每一組參數進行一次完整的 5-fold cross validation。因此對於每一組參數（例如 A），會得到五個模型 ($M_{A1}, M_{A2}, M_{A3}, M_{A4}, M_{A5}$)，接著在評估此參數時：

- confusion matrix: 將五個 fold 的結果加總
- 其他 metrics (Accuracy、Precision、Recall 等): 取平均值

請比較不同參數組合的表現並選擇最佳參數並列出每組參數在 validation 上的結果 (confusion matrix 與各項 metrics)。最後，請使用最佳參數在整個 training set 上重新訓練模型，在 test set 上進行評估，並列出此模型的 confusion matrix 與各項 evaluation metrics 的數值。

(一) 資料分割

使用問題2相同的train_set和test_set, 但不一樣的是在train_set進一步分割成5等分, 以執行5 fold cross validation, 以下為實現的程式碼。

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

for fold, (train_idx, val_idx) in enumerate(skf.split(X_train, y_train), 1):
    X_train_fold, y_train_fold = X_train[train_idx], y_train[train_idx]
    X_val_fold, y_val_fold = X_train[val_idx], y_train[val_idx]
    dump_svmlight_file(X_train_fold, y_train_fold, os.path.join(output_dir, 'cross_validation', f'train_fold{fold}.scale'))
    dump_svmlight_file(X_val_fold, y_val_fold, os.path.join(output_dir, 'cross_validation', f'val_fold{fold}.scale'))
```

(二) 訓練參數的選擇

這一部分與問題2相同

(三) 實驗結果

參數	模型表現
參數1	<pre>--- Model 1 Cross validation Test Set Evaluation --- Confusion Matrix: [[17158 1723] [1909 19210]] Accuracy: 0.9092 Precision: 0.8999 Recall: 0.9087 FPR: 0.0904 TNR: 0.9096 FNR: 0.0913</pre>
參數2	<pre>--- Model 2 Cross validation Test Set Evaluation --- Confusion Matrix: [[17056 1825] [1854 19265]] Accuracy: 0.9080 Precision: 0.9020 Recall: 0.9033 FPR: 0.0878 TNR: 0.9122 FNR: 0.0967</pre>

參數3	<pre> --- Model 3 Cross validation Test Set Evaluation --- Confusion Matrix: [[17023 1858] [1852 19267]] Accuracy: 0.9073 Precision: 0.9019 Recall: 0.9016 FPR: 0.0877 TNR: 0.9123 FNR: 0.0984 </pre>
參數4	<pre> --- Model 4 Cross validation Test Set Evaluation --- Confusion Matrix: [[17111 1770] [1823 19296]] Accuracy: 0.9102 Precision: 0.9037 Recall: 0.9063 FPR: 0.0863 TNR: 0.9137 FNR: 0.0937 </pre>
參數5	<pre> --- Model 5 Cross validation Test Set Evaluation --- Confusion Matrix: [[16516 2365] [2287 18832]] Accuracy: 0.8837 Precision: 0.8785 Recall: 0.8747 FPR: 0.1083 TNR: 0.8917 FNR: 0.1253 </pre>

在五組模型中，Model 4 表現最佳，其 Accuracy 達到 0.9102，為所有模型中最高。同時 Precision(0.9037)與 Recall(0.9063)皆維持在 0.90 以上，顯示模型在不同 fold 下仍具備穩定的分類能力。

此外，Model 4 的 False Positive Rate(FPR = 0.0863)與 False Negative Rate(FNR = 0.0937)亦相對較低，代表誤判與漏判情況皆受到良好控制。

相較之下，Model 5 表現明顯較差，其 Accuracy 僅為 0.8837，且 FPR 上升至 0.1083，顯示模型在 cross validation 下泛化能力不足。

(四) 完整訓練集重訓與測試集評估

選用的模型為參數4，因為他綜合評估下表現最好，這一部分在問題2已經有訓練和測試過了，所以沿用問題2的表現。


```
--- Model 4 Holdout Test Set Evaluation ---
Confusion Matrix:
[[4262  458]
 [ 441 4839]]
Accuracy:  0.9101
Precision: 0.9062
Recall:     0.9030
FPR:        0.0835
TNR:        0.9165
FNR:        0.0970
```

五、問題四

Question 4. (20%) ROC Curve and AUC

請使用 LIBSVM 的 `-b` (probability estimation) 選項，重新訓練前述任一模型，以取得每筆測試資料的預測機率。請自行撰寫程式：繪製 ROC curve 以及計算 AUC。(不可使用現成套件直接產生 ROC curve 或 AUC，例如 scikit-learn) 請在作業內敘述你如何繪製 ROC curve、計算 AUC 數值跟程式跑完的結果 (ROC curve 圖片)，並附上程式碼的檔案。

會使用專案中的 `roc_plot.py`，完整的程式可到專案中進行查看

(一) 資料準備

本次模型使用參數4訓練出來的模型，因為在實驗過在cross validation 和holdout都可以有最佳的正確率，我使用以下以下指令來產生model。

```
svm-train -b 1 -s 0 -t 2 -c 100 -g 0.01 data/holdout/train.scale best_model.txt
```

最後再用以下指令來取得預測出來的資料。

```
svm-predict data/holdout/test.scale best_model.txt best_model_output.txt
```

(二) 計算說明

1. ROC Curve怎麼畫

Step 1: 取得預測機率與真實標籤

從模型輸出檔 `best_model_output.txt` 讀取每筆樣本屬於正類(label=1)的機率值 `y_prob`，並從測試集 `test.scale` 讀取真實標籤 `y_true`(正類為 +1，負類為 -1)。

Step 2: 枚舉閾值

首先，建立兩個列表來分別儲存 TPR 與 FPR，並將兩者的起始值設為 0。接著，找出 `y_prob` 中所有不重複的數值，將其由大到小排序後依序作為分類閾值(threshold)。針對每一個閾值，計算對應的 TPR 與 FPR 並將結果存入列表中。最後，在兩個列表的尾端皆加入數值 1，完成所有紀錄。

以下為實現的程式碼

```
def calc_roc(y_true, y_prob):
    thresholds = sorted(set(y_prob), reverse=True)

    P = np.sum(y_true == 1)
    N = np.sum(y_true == -1)

    tprs = [0.0]
    fprs = [0.0]

    for thresh in thresholds:
        y_pred = np.where(np.array(y_prob) >= thresh, 1, -1)
        TP = np.sum((y_pred == 1) & (y_true == 1))
        FP = np.sum((y_pred == 1) & (y_true == -1))
        tprs.append(TP / P)
        fprs.append(FP / N)

    tprs.append(1.0)
    fprs.append(1.0)
    return np.array(fprs), np.array(tprs)
```

Step 3:繪圖

將所有 (FPR, TPR) 點依序連線，即得 ROC Curve。同時繪製對角虛線代表隨機分類器的基準線。

2. AUC計算說明

本程式使用梯形法(Trapezoidal Rule)數值積分計算 AUC，需使用之前在繪製 ROC Curve所計算到TPR列表和FPR列表，每一段以相鄰兩點的 FPR 差值為底、TPR 平均值為高，加總所有梯形面積即為 AUC。

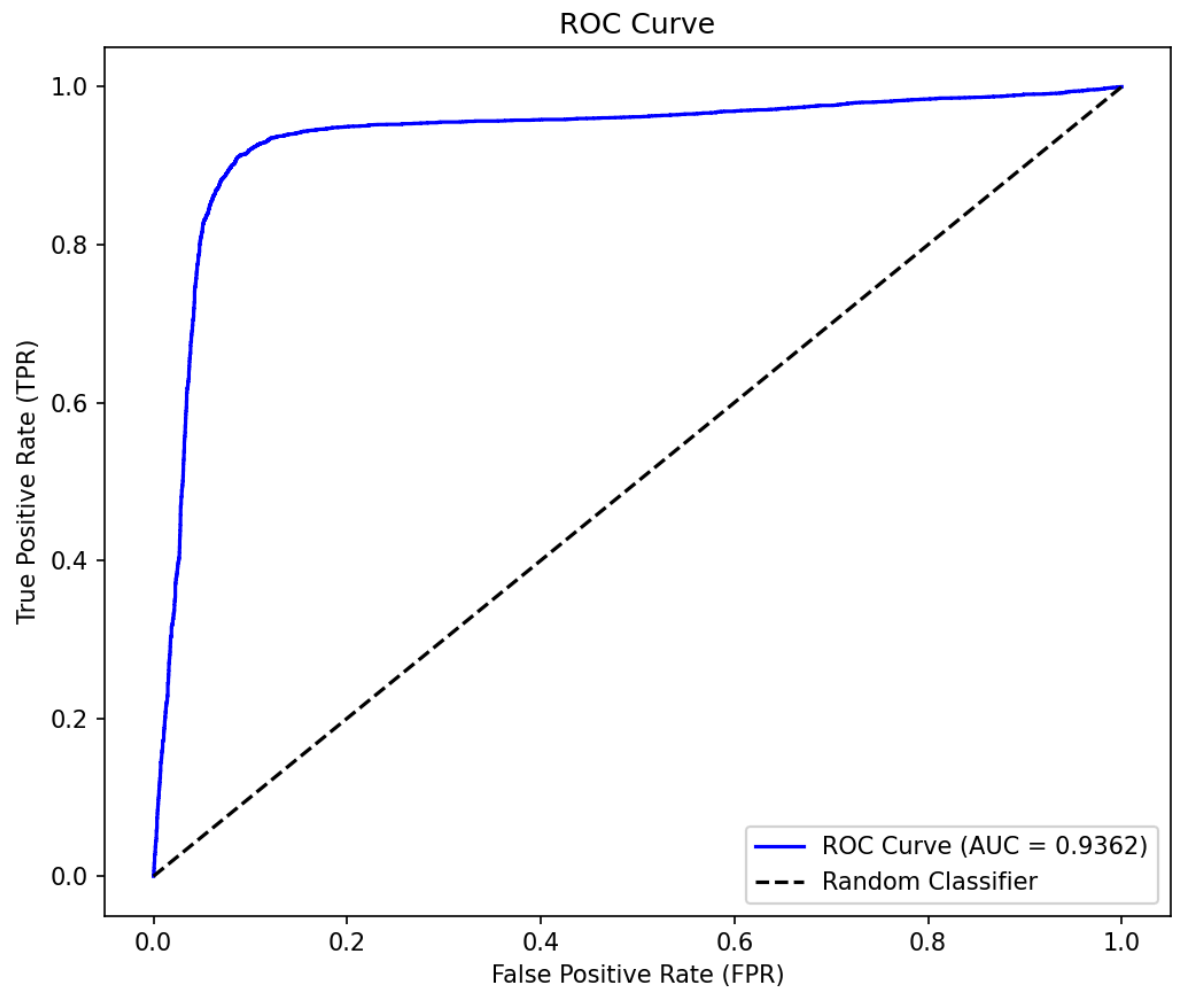
以下為實現的程式碼

```
def calc_auc(fprs, tprs):
    auc = 0.0
    for i in range(1, len(fprs)):
        dx = fprs[i] - fprs[i-1]
        avg_y = (tprs[i] + tprs[i-1]) / 2
        auc += dx * avg_y
    return auc

fprs, tprs = calc_roc(y_true, y_prob)
auc = calc_auc(fprs, tprs)
```

(三) 實驗結果

1. ROC Curve



2. AUC

計算結果

```
D:\data_analysis_hw2>python roc_plot.py  
AUC: 0.9362
```